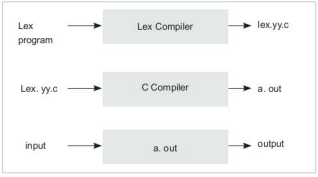

Figure 1: How the system works

It can be thought of as resulting in printing 'int' as the keyword and 'imacy' as the string. But since Lex carries out the action for the longest possible match, 'intimacy' will be identified as the string.

User defined subroutines: This section includes the definition of the *main( )* function and other user defined functions which will be copied to the end of the generated C file. The definition of *main()* is optional, since it is defined by the Lex compiler.

Program execution

A Lex file can be saved with the *.l* or *.lex* extension. Once the program is compiled using a Lex compiler, a C file *lex.yy.c* is generated. It is then compiled to get an *a.out* file, which accepts the input and produces the result. Diagramatic representation of the same is shown in Figure 1.

The following command is used to compile a program using the Lex compiler:

```
lex file_name.l
```

The C file generated -- *lex.yy.c* -- can be compiled using a gcc compiler:

```
gcc lex.yy.c -lfl
```

The above code generates the executable file *a.out*. The *lfl* or *ll* (depending on the alternatives to Lex being used) options are used to include the library functions provided by the Lex compiler.

```
./a.out < input_file_name
```

The above command will give the desired output.

Sample program 1

The following is a Lex program to count the number of characters in a file:

```
    %{
      int charCount;
    %}
//This comment is to tell that following is the rule section
    %%
    [a-zA-Z.] { charCount++;}
    %%
    main()
    {
      yylex();
      printf("%d  \n", charCount);
    }
Sample input_1:  abs. +as
Sample output_1: +7
Sample input_2:  abs.as
Sample output_2: 6
```

All the user defined variables must be declared. In this example, the variable charCount is declared in the definition section. Otherwise, when the *lex.yy.c* file is compiled, the C compiler will report the error 'charCount undeclared'. The regular expression [a-zA-Z.] represents any letter in the English alphabet or a . (dot). Whenever a letter or . (dot) is encountered, charCount is incremented by one. It is to be noted that when you want to use . (dot) literally outside [ ], not as a wild card character, put a \(backslash) before it.

When you write the rule section, make sure that no extra white space is used. Unnecessary white space will cause the Lex compiler to report an error.

In the *main()* function, *yylex()* is being called, which reads each character in the input file and is matched against the patterns specified in the rule section. If a match is found, the corresponding action will be carried out. If no match is found, that character will be written to the output file or output stream. In this program, the action to be performed when '+' is encountered is not specified in the rule section. So it is written directly to the output stream. Hence sample output_1.

The *lex.yy.c* file

The following is a description of the generated C file *lex.yy.c*. The Lex compiler will generate static arrays that serve the purpose of a finite automata, using the patterns specified in the rule section. A switch-case statement corresponding to the rule section can be seen in the file *lex.yy.c*. The following code segments are from *lex.yy.c*, obtained by compiling sample program 1.

```
switch ( yy_act )
      { /* beginning of action switch */
            case 0: /* must back up */
            /* undo the effects of YY_DO_BEFORE_ACTION */
            *yy_cp = (yy_hold_char);
            yy_cp = (yy_last_accepting_cpos);
            yy_current_state = (yy_last_accepting_state);
            goto yy_find_action;
case 1:
YY_RULE_SETUP
```